

Drone Coverage Path Planning

Lavinia Amorosi^a, Paolo Dell’Olmo^a, Justo Puerto^b, Carlos Valverde^{b,*}

^a*Department of Statistical Sciences, Sapienza University of Rome, Italy*

^b*Department of Statistical Sciences and Operational Research, University of Seville, Spain*

Abstract

We present a mixed-integer quadratic programming (MIQP) formulation for a drone coverage path planning problem. A single drone, with limited energy, must depart from a depot, cover a set of disjoint polygonal regions in the plane, and return to the depot to swap the battery and start a new tour, until all the regions are covered. Each region can be inspected at a discrete set of altitudes, each associated with a polygonal chain that defines the coverage pattern (altitude selection). A finite set of entry/exit points is associated with each region, allowing the drone to interrupt the coverage of a region and resume it later from a different point. The formulation embeds altitude selection, the geometry of the entry/exit points along the chain, and operation splitting under the endurance budget, using second-order-cone constraints for the Euclidean distances and a big-M linearization of the bilinear products between routing and distance/energy variables. We also propose an equivalent edge-based reformulation that replaces vertex-visitation variables by edge-traversal variables and uses DFJ lazy subtour elimination, and a multi-phase constructive heuristic that produces feasible solutions in under 0.1s on all tested instances and is used to warm-start the exact solver. Preliminary computational results on a testbed of artificial instances compare the heuristic against the exact solver with and without warm start.

Keywords: Drone coverage path planning, Mixed-Integer Quadratic Programming, UAV, Coverage patterns

1. Introduction

The optimization of unmanned aerial vehicle (UAV) operations is inherently multifaceted: it involves path planning, resource allocation, energy management, and collision avoidance in increasingly crowded airspaces. Determining the optimal route for a drone to cover a designated area while minimizing energy consumption and avoiding obstacles is a complex combinatorial and continuous optimization problem. When multiple drones are deployed simultaneously, coordination and scheduling add a further layer of difficulty, demanding algorithms that can handle dynamic environmental conditions and regulatory constraints.

Drones have been successfully adopted in many contexts, including disaster management [? ?], telecommunications for providing 5G coverage of rural zones [?], inspection of planar graphs [? ? ?], 3D mapping and video surveillance [?], and crop monitoring in agriculture [?]. A broader survey of civil UAV applications is given in [?].

*Corresponding author

In this paper we focus on coverage of polygonal areas in the plane by a single drone. We propose a mixed-integer quadratic programming (MIQP) formulation that jointly optimizes the altitude, the entry and exit points, and the sequence of operations under a limited energy budget. Unlike existing two-stage approaches that first plan intra-region paths and then solve an inter-region routing problem, our formulation allows the drone to interrupt the coverage of a region and resume it later, potentially reducing total energy consumption. To tackle large instances, we also develop a multi-phase constructive heuristic that provides high-quality feasible solutions in well under a second and can be used to warm-start the exact solver. The main contributions of this paper are:

- A MIQP formulation for the drone coverage path planning problem that integrates altitude selection, variable entry/exit points, and operation splitting under endurance constraints.
- An equivalent edge-based reformulation that replaces vertex-visitation variables with edge-coverage variables and uses DFJ lazy subtour elimination, offering a different trade-off between constraint structure and relaxation strength.
- A multi-phase heuristic combining chain selection, region-level tour construction, greedy endurance-aware tour splitting, coordinate-descent entry-point optimization, and intra/inter-operation local search.
- A computational study on a testbed of artificial instances with varying numbers of regions, altitude levels, and endurance values, comparing the exact solver with and without warm-start against the standalone heuristic and the two equivalent MIQP formulations.

2. State of the art

The coverage path planning problem (CPP) has been studied widely in the literature. A comprehensive survey that includes a taxonomy of CPP algorithms and drone-specific characteristics is presented in [?]. Recent work can be grouped into three broad methodological streams.

Back-and-forth coverage patterns. Several authors adopt variants of the back-and-forth (also known as lawnmower or boustrophedon) strategy for intra-region coverage. [?] propose an improved back-and-forth algorithm with a straight-turn mode to cover rectangular regions containing forbidden areas. [?] develop a mathematical model and heuristic for energy-efficient multi-UAV multi-region CPP, using back-and-forth patterns with Bezier-curve smoothing to reduce energy consumption from turns; their optimization proceeds in two decoupled phases (intra-region path generation then inter-region routing). Similarly, [?] generate waypoints for polygonal areas and apply smoothing to reduce flight time and memory footprint.

Two-stage and sequential approaches.. Many methods first plan coverage paths within each region and then solve a separate routing problem. [?] present a CPP method for spraying drones that disinfect areas, tested on realistic scenarios. [?] propose a two-stage method for mixed regions with limited energy: they shrink perimeters to generate optimal paths, then use dynamic programming to segment the overall path among multiple drones to satisfy energy limits. [?] develop a trajectory and velocity control approach for energy-efficient visual coverage of multiple terrestrial regions, solving the problem in four sequential stages.

Energy-aware formulations with altitude and speed control.. The works most closely related to ours jointly consider energy, altitude, and speed decisions. [?] allow the UAV to change both velocity and altitude during its tour, optimizing visual coverage quality subject to energy constraints. Our approach differs from all of the above in that we allow the drone to interrupt coverage of a region and move to another one before returning, rather than fully covering each region in a single pass. This flexibility, enabled by the edge set $\mathcal{E}^{ext}$ in our model, can lead to shorter total tours compared to the two-stage paradigm, as illustrated in Section 3.

3. Problem Description

In this paper we study a drone coverage path planning problem. Given a set of non-overlapping polygonal areas to be visited, a single drone with limited endurance, and a depot, the problem consists of finding the set of tours (operations) starting and ending at the depot that the drone must perform in order to cover all areas. We assume that every region can be fully covered in a single operation, i.e. the drone has enough endurance to depart from the depot, cover a region, and return to the depot in one go.

Let $\mathcal{R} = \{1, \dots, R\}$ be the set of regions to be covered. For each region $r \in \mathcal{R}$, the user specifies a set of polygonal chains $\mathcal{T}_r$, each at a different altitude and each defining a candidate coverage pattern. The visit of a region can be performed with interruptions: the drone may leave a region before completing its coverage and resume it from another point if doing so reduces the total energy consumption. The polygonal chain of a region can be traversed in both directions. For tractability, we assume that the number of interruptions per region is at most $R - 1$, where R is the number of regions.

For each region $r \in \mathcal{R}$, we index the set of $2R$ candidate points along its selected chain by $\mathcal{I}_r = \{1, \dots, 2R\}$, partitioned into four subsets:

- $\mathcal{I}_S^r = \{1, 2\}$: the two endpoints at the start of the chain (each can act as a launching or retrieving point);
- $\mathcal{I}_L^r = \{3, 5, \dots, 2R - 1\}$: the $R - 1$ launching points;
- $\mathcal{I}_R^r = \{4, 6, \dots, 2R\}$: the $R - 1$ retrieving points;
- $\mathcal{I}_T^r = \{2R + 1, 2R + 2\}$: the two endpoints at the end of the chain (each can act as a launching or retrieving point).

The launch and retrieve points come in pairs: a launch point $2k + 1$ is paired with the retrieve point $2k + 2$ for $k = 1, \dots, R - 1$.

We define the graph $\mathcal{G} = (\mathcal{V} = \{v_0\} \cup \mathcal{V}', \mathcal{E} = \mathcal{E}^{int} \cup \mathcal{E}^{ext})$, where:

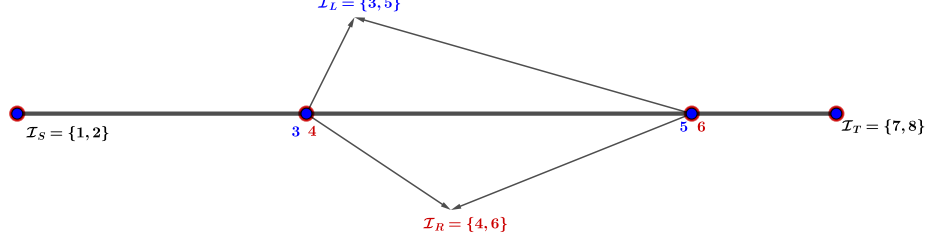


Figure 1: Indexing of the candidate points along the chain of a single region (R regions in total).

- $v_0 = (0, 0, 0)$ denotes the depot.
- $\mathcal{V}' = \bigcup_{r \in \mathcal{R}} \{r\} \times \mathcal{I}_r$ is the set of all candidate points across regions. It is partitioned as $\mathcal{V}' = \mathcal{V}_S \cup \mathcal{V}_L \cup \mathcal{V}_R \cup \mathcal{V}_T$, where $\mathcal{V}_S = \bigcup_{r \in \mathcal{R}} \{r\} \times \mathcal{I}_S^r$, and similarly for $\mathcal{V}_L$, $\mathcal{V}_R$, $\mathcal{V}_T$.
- $\mathcal{E}^{int} = \{(v, v') : v = (r, i), v' = (r, i + 1), \forall r \in \mathcal{R}, \forall i \in \mathcal{I}_r \setminus \{2R\}\}$ is the set of edges joining consecutive points on the same chain. It is partitioned into $\mathcal{E}_{LR}^{int}$ (forward edges) and $\mathcal{E}_{RL}^{int}$ (backward edges).
- $\mathcal{E}^{ext} = \{(v, v') : v = (r, i) \in \mathcal{V}' \setminus \mathcal{V}_R, v' = (r', i') \in \mathcal{V}' \setminus \mathcal{V}_L, r \neq r'\}$ is the set of arcs that join a non-retrieving point in one region to a non-launching point in another region.
- $\mathcal{O}$ is the set of operations, each a drone tour starting and ending at the depot v_0 .

For each region $r \in \mathcal{R}$, the set $\mathcal{T}_r$ of polygonal chains contains the candidate coverage patterns for that region. Each chain $t \in \mathcal{T}_r$ is associated with an altitude h_t and a sequence of segments $\{[Q^s, Q^{s+1}] : s \in \mathcal{S}_t\}$, where Q^s, Q^{s+1} are the endpoints of segment s and $\mathcal{S}_t = \{1, \dots, S_t\}$ is the set of segments of the chain. The selection of one chain per region is a decision variable of the model.

We assume that air density, denoted by ρ , depends on height as given by the following function (derived from the ideal gas law and the International Standard Atmosphere model):

$$\rho_h = \frac{p_0 M}{C_R T_0} \left(1 - \frac{Lh}{T_0}\right)^{\frac{gM}{C_R L} - 1},$$

where the constants are:

- $p_0 = 101325$ Pa: sea level standard atmospheric pressure;
- $T_0 = 288.15$ K: sea level standard temperature;

- $g = 9.80665 \text{ m/s}^2$: earth-surface gravitational acceleration;
- $L = 0.0065 \text{ K/m}$: temperature lapse rate;
- $C_R = 8.31446 \text{ J/(mol}\cdot\text{K)}$: universal gas constant;
- $M = 0.0289652 \text{ kg/mol}$: molar mass of dry air.

We also assume that the wind has a constant direction represented by the unit vector $\vec{u}^w$. The magnitude of the wind speed depends on height following the power law of [?]:

$$\nu_h^w = \nu_{10}^w \left(\frac{h}{10} \right)^a,$$

where ν_{10}^w is the wind speed at 10 m above ground and a is the Hellmann exponent (which depends on the coastal location, terrain roughness, and air stability). The wind velocity vector at height h is then $\vec{\nu}_h^w = \nu_h^w \vec{u}^w$.

Given the wind vector at altitude h , the drone airspeed required to maintain a ground velocity $\vec{\nu}$ is

$$\vec{\nu}_h^d = \vec{\nu} - \vec{\nu}_h^w.$$

Because the drone’s position and the chosen chain are decision variables, the actual flight speed along an inter-region leg is unknown before the model is solved. To avoid bilinear terms, we approximate the direction of an inter-region leg (r, r') by the unit vector from the centroid of r to the centroid of r' , giving a pre-computable ground speed $\vec{\nu}_{rr'h}^d$ (and hence energy) per region pair. The exact leg directions are recovered through the continuous position variables in the formulation.

One key feature of our problem setting is that the drone is not forced to fully explore one region before moving to another one. The edge set $\mathcal{E}^{ext}$ allows the drone to leave a region before completing its coverage and explore a different one. Other CPP contributions propose a two-stage approach, in which each region is fully covered first and then an inter-region routing problem is solved; this restriction can be suboptimal, as illustrated by the following example.

Illustrative example.. Consider an instance with one large regular polygon of n edges of length l (a single region) and n unit-size satellite regions placed externally at unit distance from each vertex of the polygon. Figure 2 shows the case $n = 3$. Assume that the cost of covering the polygon is the same under both approaches. In our approach, the drone can leave the polygon at each vertex, cover the adjacent satellite, and resume the polygon. The satellite-covering cost is then $4n$ (unit inward detour, outward to satellite, back, unit back to polygon). In the two-stage approach, the satellite regions must be covered by separate trips from the depot; the best inter-region travel has cost at least $(n - 1)l$, where l is the polygon edge length. The ratio of the two cost gaps is then $(n - 1)l/(4n)$, which grows with l . The worst case for the two-stage gap among regular polygons is the triangle ($n = 3$), giving a ratio of $l/6$.

It should be noted that our approach does not *force* the drone to leave a region before completing it; it merely *allows* this whenever doing so reduces the total cost.

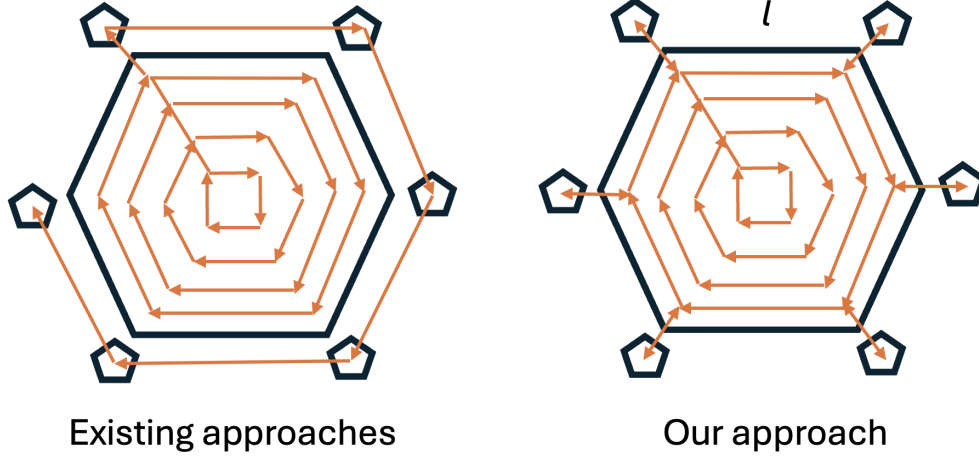


Figure 2: Comparison of the proposed approach (left) and the two-stage approach (right) for a triangle with three satellite regions.

4. Mathematical Formulation

This section presents a mixed-integer quadratic programming formulation for the problem outlined in Section 3. The model has four families of constraints: (i) location constraints, which determine the placement of the entry/exit points along the polygonal chain of each region; (ii) drone path constraints, which govern the sequence of operations and eliminate subtours; (iii) distance constraints, which model the movements between points; and (iv) consumption constraints, which translate distances into energy and enforce the endurance budget. The decision variables used in the formulation are summarized in Tables 1 and 2.

Table 1: Binary and integer variables

Name	Domain	Range	Description
μ_v^{ts}	$\mathcal{V}' \times \{(t, s) : t \in \mathcal{T}_r, s \in \mathcal{S}_t\}$	$\{0, 1\}$	1, if vertex $v = (r, i)$ is located at segment s of chain t .
α_t	$\{t \in \mathcal{T}_r : r \in \mathcal{R}\}$	$\{0, 1\}$	1, if polygonal chain t is selected.
$x_{vv'}^o$	$\mathcal{E} \times \mathcal{O}$	$\{0, 1\}$	1, if the drone traverses edge (v, v') in operation o .
y_v^o	$\mathcal{V}' \times \mathcal{O}$	$\{0, 1\}$	1, if vertex v is visited during operation o .
ζ^o	$\mathcal{O}$	$\{0, 1\}$	1, if all vertices are visited before starting operation o .
k^o	$\mathcal{O}$	$\{1, \dots, \mathcal{V}' \}$	number of vertices visited in operation o .
$\varphi_{rr'}$	$\{(r, r') \in \mathcal{R} \times \mathcal{R} : r \neq r'\}$	$\{0, 1\}$	1, if the drone moves from r to r' at the height of r .
$\psi_{rr'}$	$\{(r, r') \in \mathcal{R} \times \mathcal{R} : r \neq r'\}$	$\{0, 1\}$	1, if the height of r is lower than that of r' .

4.1. Location constraints

For each vertex $v = (r, i) \in \mathcal{V}'$ we define its spatial coordinates $P_v = (P_v^x, P_v^y, P_v^z) \in \mathbb{R}^3$. Let μ_v^{ts} be a binary variable that is one if vertex v is placed on segment s of chain t , and let $\gamma_v \in [0, 1]$ denote the relative position within that segment. The product $\gamma_v \cdot \mu_v^{ts}$ is linearized via the auxiliary continuous variable η_v^{ts} , constrained by standard Fortet inequalities ($0 \leq \eta_v^{ts} \leq \mu_v^{ts}$, $\gamma_v - (1 - \mu_v^{ts}) \leq \eta_v^{ts} \leq \gamma_v$). The coordinates of v are then given by:

$$P_v = \sum_{t \in \mathcal{T}_r} \sum_{s \in \mathcal{S}_t} [\mu_v^{ts} Q^s + \eta_v^{ts} (Q^{s+1} - Q^s)], \quad \forall v = (r, i) \in \mathcal{V}', \quad (\text{Loc-C}_1)$$

Table 2: Continuous variables

Name	Domain	Range	Description
P_v	$\mathcal{V}'$	$\mathbb{R}^3$	Coordinates of the point associated to vertex v .
γ_v	$\mathcal{V}'$	$[0, 1]$	Position of vertex v within its selected segment.
λ_v	$\mathcal{V}'$	$[0, S_t] : t \in \mathcal{T}_r$	Global parametrization of vertex v on the selected chain.
η_v^{ts}	$\mathcal{V}' \times \{(t, s) : t \in \mathcal{T}_r, s \in \mathcal{S}_t\}$	$[0, 1]$	Linearization of $\gamma_v \cdot \mu_v^{ts}$.
df_{s_v}	$\mathcal{V}'$	$[0, L_{\max}]$	Cumulative distance from the start of the chain to vertex v .
ef_{s_v}	$\mathcal{V}'$	$[0, W_{\max}]$	Cumulative energy from the start of the chain to vertex v .
$d_{vv'}$	$\mathcal{E}$	$\mathbb{R}^+$	Distance between vertex v and vertex v' .
$d_{vv'}^{xy}$	$\mathcal{E}^{ext}$	$\mathbb{R}^+$	Horizontal distance between v and v' .
$d_{vv'}^z$	$\mathcal{E}^{ext}$	$\mathbb{R}^+$	Vertical distance between v and v' .
$w_{vv'}$	$\mathcal{E}$	$\mathbb{R}^+$	Energy consumption between v and v' .
u_v^o	$\mathcal{V}' \times \mathcal{O}$	$[1, \mathcal{V}']$	Position of v in the tour of operation o (MTZ).
ℓ_{uv}^o	$\mathcal{E} \times \mathcal{O}$	$[0, \infty)$	Linearization of $x_{uv}^o \cdot d_{uv}$ (objective) and $x_{uv}^o \cdot w_{uv}$ (endurance).
ρ_t	$\{t \in \mathcal{T}_r : r \in \mathcal{R}\}$	$[0, \rho(0)]$	Air density at the altitude of chain t (pre-computed from h_t).

$$P_v^z = \sum_{t \in \mathcal{T}_r} \sum_{s \in \mathcal{S}_t} \mu_v^{ts} h_t, \quad \forall v = (r, i) \in \mathcal{V}', \quad (\text{Loc-C}_1^z)$$

where h_t is the altitude of chain t . Constraint (Loc-C₁^z) defines the vertical coordinate independently, since each chain t is associated with a single flight altitude. The segment assignment and chain selection are enforced by:

$$\sum_{s \in \mathcal{S}_t} \mu_v^{ts} = \alpha_t, \quad \forall v = (r, i) \in \mathcal{V}', \forall t \in \mathcal{T}_r, \quad (\text{Loc-C}_2)$$

$$\sum_{t \in \mathcal{T}_r} \alpha_t = 1, \quad \forall r \in \mathcal{R}, \quad (\text{Loc-C}_3)$$

where α_t is a binary variable that is one if chain t is selected. The parametrization of vertices along the chain is defined via the continuous variable $\lambda_v \in [0, S_t]$:

$$\lambda_v = \sum_{t \in \mathcal{T}_r} \sum_{s \in \mathcal{S}_t} (s - 1) \mu_v^{ts} + \gamma_v, \quad \forall v \in \mathcal{V}', \quad (\text{Loc-C}_4)$$

$$\lambda_v = 0, \quad \forall v = (r, 1), (r, 2), \quad (\text{Loc-C}_5)$$

with the following fixing at the chain endpoints (start vertices $(r, 1), (r, 2)$ and end vertices $(r, 2R + 1), (r, 2R + 2)$):

$$\mu_v^{t0} = \alpha_t, \quad \forall v = (r, 1), (r, 2), \forall t \in \mathcal{T}_r, \quad (\text{Loc-C}_{5b})$$

$$\lambda_u \leq \lambda_v, \quad \forall (u, v) \in \mathcal{E}_{RL}^{int} \cup \mathcal{E}_{RL}^S \cup \mathcal{E}_{RL}^T, \quad (\text{Loc-C}_6)$$

$$\lambda_u = \lambda_v, \quad \forall (u, v) \in \mathcal{E}_{LR}^{int} \cup \mathcal{E}_{LR}^S \cup \mathcal{E}_{LR}^T, \quad (\text{Loc-C}_7)$$

$$\lambda_v = \sum_{t \in \mathcal{T}_r} S_t \alpha_t, \quad \forall v = (r, 2R + 1), (r, 2R + 2), \quad (\text{Loc-C}_8)$$

$$\mu_v^{t, S_t - 1} = \alpha_t, \quad \forall v = (r, 2R + 1), (r, 2R + 2), \forall t \in \mathcal{T}_r. \quad (\text{Loc-C}_{8b})$$

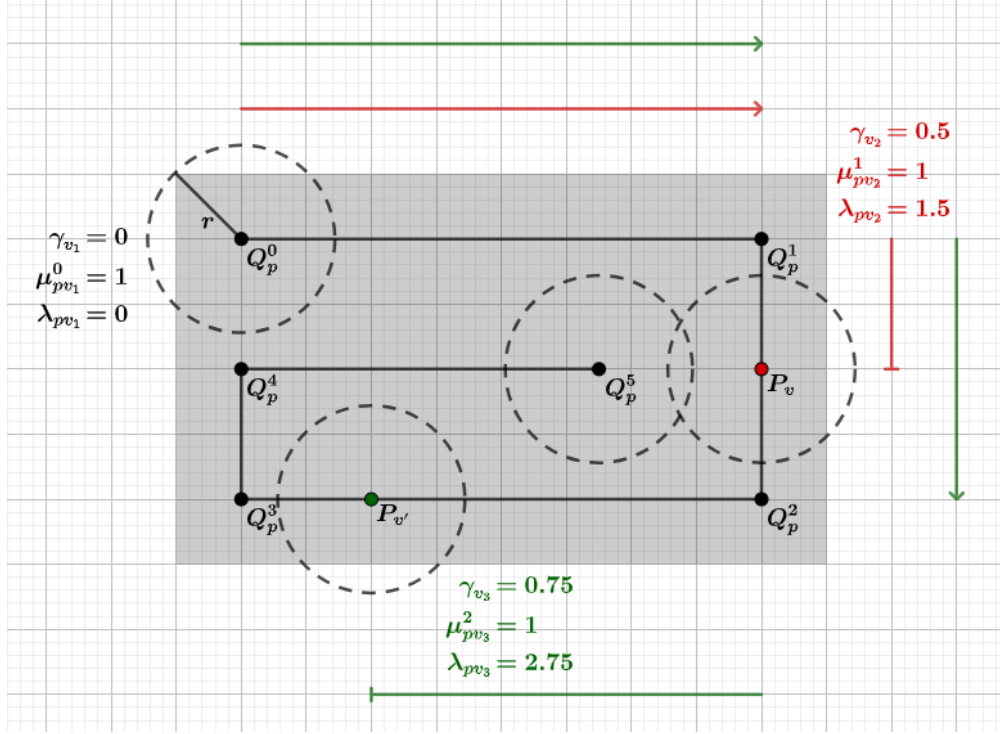


Figure 3: Parameterization of vertices

Constraint (Loc-C₄) defines λ_v as the cumulative length from the chain start to the point, expressed as the index of the selected segment (minus one) plus the offset γ_v within that segment. Constraint (Loc-C₅) fixes the parametrization at zero for the start vertices. Constraint (Loc-C_{5b}) fixes the first segment for start vertices. Constraints (Loc-C₆) and (Loc-C₇) enforce the traversal order and the coincidence of launch–retrieve pairs, respectively. Constraint (Loc-C₈) sets λ to the total chain length for the end vertices, and (Loc-C_{8b}) fixes the last segment for those vertices.

4.2. Drone path constraints

In this subsection, we describe the set of constraints related to the operations that the drone must perform to visit all vertices described in $\mathcal{V}'$. In this context, we define several families of variables that model the path. Let $x_{vv'}^o$ be a binary variable that is one if the drone goes from vertex v to vertex v' in operation o . Let y_v^o be a binary variable that is one if the vertex v is visited during the operation o . Let also ζ^o be a binary variable that is one if all vertices are visited before starting the operation o . We count the number of vertices visited in operation o by means of the integer variable k^o . Additionally, u_v^o is a continuous variable that represents the position of vertex v in the tour of operation o , used in the subtour elimination constraints.

$$\sum_{v' \in \mathcal{V}} x_{v_0 v'}^o = 1 - \zeta^o, \quad \forall o \in \mathcal{O}, \quad (D_{path} - C_1)$$

$$\sum_{v' \in \mathcal{V}} x_{v' v}^o - \sum_{v' \in \mathcal{V}} x_{v v'}^o = 0, \quad \forall v \in \mathcal{V}', \forall o \in \mathcal{O}, \quad (D_{path} - C_2)$$

$$\begin{aligned}
\sum_{v' \in \mathcal{V}} x_{v'v_0}^o &= 1 - \zeta^o, & \forall o \in \mathcal{O}, & (D_{path} - C_3) \\
\sum_{v' \in \mathcal{V}} x_{vv'}^o &= y_v^o, & \forall v \in \mathcal{V}, \forall o \in \mathcal{O}, & (D_{path} - C_4) \\
\sum_{v' \in \mathcal{V}} x_{v'v}^o &= y_v^o, & \forall v \in \mathcal{V}, \forall o \in \mathcal{O}, & (D_{path} - C_5) \\
\sum_{o \in \mathcal{O}} y_v^o &= 1, & \forall v \in \mathcal{V}', & (D_{path} - C_6) \\
u_v^o - u_{v'}^o + 1 &\leq |\mathcal{V}'|(1 - x_{vv'}^o), & \forall v, v' \in \mathcal{V}' : v \neq v', \forall o \in \mathcal{O}, & (D_{path} - C_7) \\
1 &\leq u_v^o \leq |\mathcal{V}'|, & \forall v \in \mathcal{V}', \forall o \in \mathcal{O}. & (D_{path} - C_{7b}) \\
\sum_{o \in \mathcal{O}} x_{vv'}^o &= 1, & \forall v, v' \in \mathcal{V}' : (v, v') \in \mathcal{E}_{RL}^{int} \cup \mathcal{E}_{RL}^S \cup \mathcal{E}_{RL}^T. & (D_{path} - C_8)
\end{aligned}$$

Constraints $(D_{path} - C_1)$ ensure that the drone exits from the depot if there is a vertex that is not already visited in operation o . Constraints $(D_{path} - C_2)$ are the flow conservation constraints. Constraints $(D_{path} - C_3)$ guarantee that drone operation ends in the depot if there is a vertex that has not been visited in operation o . Constraints $(D_{path} - C_4)$ and $(D_{path} - C_5)$ ensure that the vertex v is visited if there is a path passing through this vertex. Constraints $(D_{path} - C_6)$ impose that every vertex must be visited in some operation. Inequalities $(D_{path} - C_7)$ are the Miller–Tucker–Zemlin (MTZ) subtour elimination constraints [?], which use the auxiliary variables u_v^o to enforce that the tours in each operation contain no cycles that exclude the depot. The bounds $(D_{path} - C_{7b})$ define the range of these continuous variables. Finally, constraints $(D_{path} - C_8)$ guarantee that each polygonal chain is completely traversed by requiring that all edges of the path graph associated with each region are traversed in some operation.

Valid Inequalities

In this subsection, we present a set of valid inequalities for the representation of the drone path that are included in the formulation to enhance it. Let ζ^o be a binary variable that takes the value of one if all vertices have been visited before starting the operation o , and zero otherwise. If this is the case, then all vertices must also be visited before the operation $o + 1$. Therefore, the ζ variables must satisfy the following monotonicity constraint:

$$\zeta^o \leq \zeta^{o+1}, \quad \forall o \in \mathcal{O} \setminus \{O\}. \quad (\text{Monotonicity})$$

We define k^o as the number of vertices visited during the operation o , excluding the depot. The variables y can be used to determine this quantity, where we recall that y_v^o equals one if the vertex v is visited in operation o . Thus:

$$k^o = \sum_{v \in \mathcal{V}'} y_v^o, \quad \forall o \in \mathcal{O}. \quad (k - C)$$

If ζ^o is equal to one, it implies that all vertices in $\mathcal{V}'$ must have been visited in previous operations. This relationship is enforced through the following inequality:

$$\sum_{o'=1}^{o-1} k^{o'} \geq |\mathcal{V}'| \zeta^o, \quad \forall o \in \mathcal{O}. \quad (VI - C_1)$$

To refine the feasible solution space, we assume, without loss of generality, that an operation o cannot consist in remaining at the depot, if there are still targets left to visit. This requirement is ensured by the following constraint:

$$k^o \geq 1 - \zeta^o, \quad \forall o \in \mathcal{O}. \quad (VI - C_2)$$

4.3. Distance constraints

Since vertex positions are decision variables, all distances must be expressed through continuous variables. We first consider pairs of vertices belonging to the same region, then pairs in different regions.

Intra-region distances. For each region r , the polygonal chain t is composed of segments $\mathcal{S}_t$ of known lengths $L(s) = \|Q^{s+1} - Q^s\|_2$. Rather than introducing bilinear products $\mu_v^s \mu_{v'}^{s'}$, we define an auxiliary continuous variable dfs_v that accumulates the distance from the start of the chain to vertex v :

$$\begin{aligned} dfs_v &= \sum_{t \in \mathcal{T}_r} \sum_{s \in \mathcal{S}_t} [\text{cum_before}(s) \mu_v^{ts} + L(s) \eta_v^{ts}], & \forall v \in \mathcal{V}', & \quad (\text{DFS}) \\ d_{uv} &= dfs_v - dfs_u, & \forall (u, v) \in \mathcal{E}_{RL}^{int}, & \quad (\text{Dist-C}_1) \\ d_{uv} &= 0, & \forall (u, v) \in \mathcal{E}_{LR}^{int}, & \quad (\text{Dist-C}_2) \end{aligned}$$

where $\text{cum_before}(s) = \sum_{s' < s} L(s')$ is the length of the chain up to the start of segment s , and $\eta_v^{ts} = \gamma_v \cdot \mu_v^{ts}$ is the same linearization variable used in (Loc-C₁). For RL edges the distance is simply the difference of cumulative distances from the chain start; for LR edges the two vertices coincide ($\lambda_u = \lambda_v$ from (Loc-C₇)) so the distance is zero.

Inter-region distances. For vertices in different regions we decompose the movement into horizontal and vertical components. Let $d_{vv'}^{xy}$ and $d_{vv'}^z$ be continuous variables representing the horizontal and vertical distances, respectively. The total distance is then:

$$\begin{aligned} d_{vv'} &= d_{vv'}^{xy} + d_{vv'}^z, & \forall (v, v') \in \mathcal{E}^{ext}, & \quad (\text{Dist-C}_3) \\ d_{vv'}^{xy} &\geq \|(P_v^x, P_v^y) - (P_{v'}^x, P_{v'}^y)\|_2, & \forall (v, v') \in \mathcal{E}^{ext}, & \quad (\text{Dist-C}_4) \\ d_{vv'}^z &\geq P_v^z - P_{v'}^z, & \forall (v, v') \in \mathcal{E}^{ext}, & \quad (\text{Dist-C}_5) \\ d_{vv'}^z &\geq P_{v'}^z - P_v^z, & \forall (v, v') \in \mathcal{E}^{ext}. & \quad (\text{Dist-C}_6) \end{aligned}$$

Constraint (Dist-C₄) is a second-order cone constraint that computes the Euclidean horizontal distance. Constraints (Dist-C₅)–(Dist-C₆) give the absolute vertical difference via a standard linearization.

4.4. Consumption constraints

The energy consumed by the drone is modelled using the drag formula $w = \frac{1}{2}Ac\rho\nu d$, where A is the frontal area, c the drag coefficient, ρ the air density, ν the drone speed, and d the travelled distance. We define the constants $E^{xy} = E^z = \frac{1}{2}cA$ for brevity. As with distances, we treat intra-region and inter-region pairs separately.

Intra-region energy. Following the same approach as for distances, we introduce an auxiliary continuous variable efs_v that accumulates the energy from the start of the chain to vertex v :

$$efs_v = \sum_{t \in \mathcal{T}_r} \sum_{s \in \mathcal{S}_t} C_{ts} [\text{cum_before}(s) \mu_v^{ts} + L(s) \eta_v^{ts}], \quad \forall v \in \mathcal{V}', \quad (\text{EFS})$$

$$w_{uv} = efs_v - efs_u, \quad \forall (u, v) \in \mathcal{E}_{RL}^{int}, \quad (w\text{-C}_1)$$

$$w_{uv} = 0, \quad \forall (u, v) \in \mathcal{E}_{LR}^{int}, \quad (w\text{-C}_2)$$

where $C_{ts} = E^{xy} \rho_t \nu_{ts}^d$ is the energy per unit distance on segment s of chain t , computed as a constant during pre-processing. The drone ground speed ν_{ts}^d along segment s is obtained from $\bar{\nu}_{ts}^d = v_{\text{cruise}} \frac{Q^{s+1} - Q^s}{\|Q^{s+1} - Q^s\|} - \bar{\nu}_h^w$ for forward traversal (and the opposite sign for backward traversal).

Inter-region energy. For vertices in different regions, the movement involves both horizontal and vertical components at possibly different altitudes. Let $\varphi_{rr'}$ be a binary variable that is one if the drone departs from region r at the altitude selected for that region; let $\psi_{rr'}$ be one if the altitude of r is lower than that of r' . The energy consumption is then:

$$\begin{aligned} w_{vv'} = & \varphi_{rr'} \left(E^{xy} d_{vv'}^{xy} \sum_{t \in \mathcal{T}_r} \alpha_t \rho_t |\bar{\nu}_{rr't}^d| + E^z \psi_{rr'} d_{vv'}^z \frac{1}{2} \sum_{\tau \in \{r, r'\}} \sum_{t \in \mathcal{T}_\tau} \alpha_t \rho_t \nu^z \right) \\ & + (1 - \varphi_{rr'}) \left(E^{xy} d_{vv'}^{xy} \sum_{t \in \mathcal{T}_{r'}} \alpha_t \rho_t |\bar{\nu}_{rr't}^d| + E^z (1 - \psi_{rr'}) d_{vv'}^z \frac{1}{2} \sum_{\tau \in \{r, r'\}} \sum_{t \in \mathcal{T}_\tau} \alpha_t \rho_t \nu^z \right), \quad (w\text{-C}_3) \end{aligned}$$

for all $(v, v') \in \mathcal{E}^{ext}$ with $v = (r, i)$, $v' = (r', i')$. All bilinear products involving $\varphi_{rr'}$, $\psi_{rr'}$, and α_t are linearized explicitly using standard Fortet constraints.

Depot energy. For movements to and from the depot $v_0 = (0, 0, 0)$, we use a simplified model with constant sea-level air density ρ_0 and a constant horizontal cruise speed ν^{xy} . The depot-leg distance is decomposed as:

$$d_{v_0v} = d_v^{xy} + d_v^z, \quad \forall v \in \mathcal{V}', \quad (\text{Dep-C}_1)$$

$$d_v^{xy} \geq \|(P_v^x, P_v^y)\|_2, \quad \forall v \in \mathcal{V}', \quad (\text{Dep-C}_2)$$

$$d_v^z \geq |P_v^z|, \quad \forall v \in \mathcal{V}', \quad (\text{Dep-C}_3)$$

$$w_{v_0v} = E^{xy} \rho_0 \nu^{xy} d^{xy} + \frac{1}{2} E^z \rho_0 \nu^z d^z, \quad \forall v \in \mathcal{V}', \quad (\text{Dep-C}_4)$$

$$w_{vv_0} = E^{xy} \rho_0 \nu^{xy} d^{xy}, \quad \forall v \in \mathcal{V}'. \quad (\text{Dep-C}_5)$$

The ascent energy (Dep-C₄) includes both horizontal and vertical work; the descent (Dep-C₅) assumes the drone recovers no energy (vertical contribution is zero).

4.5. Endurance constraint

The bilinear product $x_{uv}^o w_{uv}$ in the endurance constraint (and $x_{uv}^o d_{uv}$ in the objective) is linearized via auxiliary continuous variables ℓ_{uv}^o with the following big-M formulation:

$$\ell_{uv}^o \leq x_{uv}^o \cdot M, \quad \forall (u, v) \in \mathcal{E}, \forall o \in \mathcal{O}, \quad (\text{End-Lin}_1)$$

$$\ell_{uv}^o \leq w_{uv}, \quad \forall (u, v) \in \mathcal{E}, \forall o \in \mathcal{O}, \quad (\text{End-Lin}_2)$$

$$\ell_{uv}^o \geq w_{uv} - (1 - x_{uv}^o)M, \quad \forall (u, v) \in \mathcal{E}, \forall o \in \mathcal{O}, \quad (\text{End-Lin}_3)$$

$$\ell_{uv}^o \geq 0, \quad \forall (u, v) \in \mathcal{E}, \forall o \in \mathcal{O}, \quad (\text{End-Lin}_4)$$

where M is a sufficiently large constant. The endurance of each operation is then enforced by:

$$\sum_{(u,v) \in \mathcal{E}} \ell_{uv}^o \leq W^{\max}, \quad \forall o \in \mathcal{O}. \quad (\text{Endurance})$$

4.6. Objective function

The objective is to minimize the total distance travelled across all operations, equivalently the total flight time since the drone speed is constant per segment. Using the same linearization as above,

$$\min W = \sum_{(u,v) \in \mathcal{E}} \sum_{o \in \mathcal{O}} \ell_{uv}^o, \quad (\text{Objective})$$

where ℓ_{uv}^o now linearizes $x_{uv}^o \cdot d_{uv}$ (the same variables can be reused because a single ℓ_{uv}^o per edge-operation pair suffices, and each ℓ_{uv}^o is constrained by both d_{uv} and w_{uv} via the big-M formulation above).

4.7. Formulation

The complete formulation of the CPP follows:

$$\begin{aligned} & \text{minimize} \quad W = \sum_{(u,v) \in \mathcal{E}} \sum_{o \in \mathcal{O}} \ell_{uv}^o && (\text{Formulation}) \\ & \text{subject to} \quad (D_{\text{path}} - C_1) - (D_{\text{path}} - C_{7b}), \\ & \quad (\text{Monotonicity}), (k - C), (VI - C_1) - (VI - C_2), \\ & \quad (\text{Loc-C}_1) - (\text{Loc-C}_{8b}), \\ & \quad (\text{DFS}), (\text{Dist-C}_1) - (\text{Dist-C}_6), \\ & \quad (\text{EFS}), (w - C_1) - (w - C_3), (\text{Dep-C}_1) - (\text{Dep-C}_5), \\ & \quad (\text{End-Lin}_1) - (\text{Endurance}) \end{aligned}$$

Table 3: Binary and integer variables for the edge-based formulation

Name	Domain	Range	Description
μ_v^s	$\mathcal{V}' \times \{s \in \mathcal{S}_t : t \in \mathcal{T}_r\}$	$\{0, 1\}$	1, if vertex $v = (r, i)$ is located at the segment $s \in \mathcal{S}_t$, where $t \in \mathcal{T}_r$.
α_t	$\{t \in \mathcal{T}_r : r \in \mathcal{R}\}$	$\{0, 1\}$	1, if polygonal chain t is selected.
$x_{vv'}^o$	$\mathcal{E} \times \mathcal{O}$	$\{0, 1\}$	1, if the drone traverses the edge (v, v') in operation o .
ζ^o	$\mathcal{O}$	$\{0, 1\}$	1, if all vertices are visited before starting operation o .
k^o	$\mathcal{O}$	$[1, 2, \dots, \mathcal{E}^{int}]$	number of intra-ring edges covered in operation o .
$\varphi_{rr'}$	$\{(r, r') \in \mathcal{R} \times \mathcal{R} : r \neq r'\}$	$\{0, 1\}$	1, if the drone moves from region r to region r' at the height selected to visit region r .
$\psi_{rr'}$	$\{(r, r') \in \mathcal{R} \times \mathcal{R} : r \neq r'\}$	$\{0, 1\}$	1, if the height selected to visit region r is lower than the height selected to visit region r' .

4.8. Alternative edge-based formulation

The formulation presented above treats each vertex in $\mathcal{V}'$ individually: location variables μ_v^s , γ_v are defined per vertex, and path variables $x_{vv'}^o$ are defined per pair of vertices. When the number of regions grows, the combinatorial explosion of vertex pairs—especially in $\mathcal{E}^{ext}$ —can make the model difficult to solve. An alternative representation can be obtained by eliminating the per-vertex visitation variables y_v^o entirely and using the traversal variables x_{uv}^o directly to count covered edges, following a perspective akin to the rural postman problem.

In this alternative formulation, the location constraints (Loc-C₁ through Loc-C₅, ρ -selection) and the distance and energy constraints (Dist-C, w -C, Endurance) remain unchanged, as they do not depend on y_v^o . The binary and integer variables for this formulation are listed in Table 3; the continuous variables are the same as in Table 2 of the vertex-based formulation, except for the MTZ auxiliary variables u_v^o which are not needed (subtour elimination is enforced via DFJ lazy cuts). The drone path constraints are modified as follows.

Drone path constraints (edge-based). The flow conservation (D_{path}-C₂) and the depot constraints (D_{path}-C₁, D_{path}-C₃) are kept unchanged. The MTZ subtour elimination (D_{path}-C₇), which relies on auxiliary vertex-position variables u_v^o , is replaced by Dantzig–Fulkerson–Johnson (DFJ) exponential subtour elimination constraints separated dynamically via a lazy constraint callback during the branch-and-cut search. At each feasible integer incumbent, a breadth-first search from the depot identifies, for each operation o , the set of vertices reachable from the depot. If a proper subset $S \subsetneq \mathcal{V}'$ that does not contain the depot forms a connected component, the following cut is added to the model:

$$\sum_{u \in S, v \notin S} x_{uv}^o \geq 1, \quad \forall S \subsetneq \mathcal{V}' : v_0 \notin S, \forall o \in \mathcal{O}. \quad (1)$$

This constraint enforces that at least one edge must leave S , thereby preventing subtours that avoid the depot. The lazy callback avoids adding the full exponential family of DFJ constraints a priori and integrates naturally with the branch-and-cut framework of modern solvers such as Gurobi.

The degree constraints are relaxed: instead of coupling each traversal to a vertex-visitation variable, we bound the number of traversals incident to each vertex:

$$\sum_{v' \in \mathcal{V}} x_{vv'}^o \leq 1, \quad \forall v \in \mathcal{V}, \forall o \in \mathcal{O}, \quad (\text{D}'_{\text{path}}\text{-C}_4)$$

$$\sum_{v' \in \mathcal{V}} x_{v'v}^o \leq 1, \quad \forall v \in \mathcal{V}, \forall o \in \mathcal{O}, \quad (\text{D}'_{\text{path}}\text{-C}_5)$$

The constraint $D_{\text{path}}\text{-}C_8$ (each intra edge traversed exactly once) is kept unchanged. Since intra-ring edges are directed (only the forward direction exists), the traversal variable x_{uv}^o already serves as a coverage indicator for the edge (u, v) , and a separate coverage variable is unnecessary.

For idle operations ($\zeta^o = 1$), all traversal variables are forced to zero via the constraint

$$\sum_{(u,v) \in \mathcal{E}} x_{uv}^o \leq (|\mathcal{V}'|+1)(1 - \zeta^o), \quad \forall o \in \mathcal{O}, \quad (D_{\text{path}}\text{-}C_9)$$

where the big-M coefficient $|\mathcal{V}'|+1$ is a tight upper bound on the number of edges in any feasible tour (at most one outgoing edge per vertex plus the depot departure). This is significantly tighter than the original bound $|\mathcal{V}|^2$.

Adapted valid inequalities. The monotonicity constraint (Monotonicity) and VI-2 ($VI - C_2$) are unchanged. The k -counting constraint and VI-1 are updated to use the edge-based variables:

$$k^o = \sum_{(u,v) \in \mathcal{E}^{int}} x_{uv}^o, \quad \forall o \in \mathcal{O}, \quad (k\text{-}C')$$

$$\sum_{o'=1}^{o-1} k^{o'} \geq |\mathcal{E}^{int}| \zeta^o, \quad \forall o \in \mathcal{O}. \quad (VI\text{-}1')$$

Equivalence. The traversal variables x_{uv}^o replace the vertex-visitation variables y_v^o in the valid inequalities: k^o counts edges traversed rather than vertices visited. In the vertex-based formulation, the degree constraints $DP_4\text{--}DP_5$ couple each vertex visit to two incident edges, so each intra-ring edge contributes exactly one to k^o via its two endpoint vertices. Here it contributes directly via x_{uv}^o , and $D_{\text{path}}\text{-}C_8$ ensures each edge is traversed exactly once. The two formulations share the same set of feasible integer solutions and therefore the same optimal value. The edge-based formulation is implemented alongside the vertex-based one in our computational framework, allowing a direct comparison of their performance.

5. Heuristic Solution Method

The mixed-integer formulation of Section 4 is too expensive to solve directly on instances with many regions or rings. We therefore complement it with a constructive heuristic that produces high-quality feasible solutions in well under a second and can be used to warm-start the exact solver. The algorithm is organized in three logical phases, summarized in Algorithm 1 and detailed below:

1. **Pre-processing (§5.1–5.1).** Decide *which* chain (altitude and shape) to use for each region, and pre-compute the geometric data of the selected rings.
2. **Construction (§5.2–5.2).** Build a first feasible solution: order the regions to visit, expand each region into a sequence of rings, and cut the resulting “giant tour” into pieces (operations) that respect the drone’s endurance.

Algorithm 1: Multi-phase heuristic for the drone CPP.

Input : Instance (regions $\mathcal{R}$, depot v_0 , drone parameters, wind)
Output: Feasible solution S (set of tours respecting endurance)

```

// Phase 1: pre-processing foreach region  $r \in \mathcal{R}$  do
|   Select best chain  $t_r^*$  ;
|                                     ▷ *[f]§5.1
end
Build flat list  $\mathcal{K}$  of rings from selected chains ;
|                                     ▷ *[f]§5.1

// Phase 2: construction Order regions by nearest-neighbour on centroids; expand
rings in index order to form  $\pi$  ;
|   ▷ *[f]§5.2 Split  $\pi$  into feasible operations  $\mathcal{P}$  respecting  $W^{\max}$  ;
|                                     ▷ *[f]§5.2

// Phase 3: improvement for  $it \leftarrow 1$  to  $N_{coord}$  do
|   Optimize every entry point ;
|                                     ▷ *[f]§5.3
end
Local search: 2-opt per operation, then Or-opt between operations ;
|   ▷ *[f]§5.3 Build and return  $S$  ;
|                                     ▷ *[f]§5.4

```

3. **Improvement (§5.3–5.4).** Polish the feasible solution: choose the best point on each ring to enter and exit (§5.3), and run local-search moves that re-order rings within an operation or move them between operations (§5.3).

5.1. Pre-processing

Chain selection

Given a set of candidate chains $\mathcal{T}_r$ for each region $r \in \mathcal{R}$, the heuristic must select exactly one chain per region to freeze the coverage pattern and altitude. Selecting more rings (lower altitude) yields finer coverage but increases the total flight distance; selecting fewer rings (higher altitude) reduces the flight distance at the expense of resolution. The choice is made independently per region using a two-step rule:

1. **Filter by detail.** Among the candidate chains of region r , discard any chain that is strictly less detailed than another, i.e. those whose ring count $|\mathcal{K}_t|$ is not the maximum among all candidates for that region. This ensures the finest available coverage resolution within each region.
2. **Pick by proxy cost.** From the remaining candidates, select the one minimizing

$$c(t) = \sum_{\kappa \in \mathcal{K}_t} \text{perim}(\kappa) + \frac{1}{2}(d_{xy}(\text{centroid}(r), v_0) + h_t),$$

Algorithm 2: Chain selection per region.

Input : Regions $\mathcal{R}$, depot v_0 , chains $\{\mathcal{T}_r\}_{r \in \mathcal{R}}$
Output: Chain selection $s : \mathcal{R} \rightarrow \mathbb{N}$

```

foreach region  $r \in \mathcal{R}$  do
     $t^* \leftarrow 0, b \leftarrow \infty$ ;
    foreach chain  $t \in \mathcal{T}_r$  do
        if  $|\mathcal{K}_t| < \max_{t' \in \mathcal{T}_r} |\mathcal{K}_{t'}|$  then
            | continue
        end
         $c \leftarrow \sum_{\kappa \in \mathcal{K}_t} \text{perim}(\kappa) + \frac{1}{2}(d_{xy}(\text{centroid}(r), v_0) + h_t)$ ;
        if  $c < b$  then
            |  $t^* \leftarrow t, b \leftarrow c$ 
        end
    end
     $s[r] \leftarrow t^*$ 
end

```

where $\text{perim}(\kappa)$ is the perimeter of ring κ , $\text{centroid}(r)$ is the centroid of region r , d_{xy} is the planar Euclidean distance, and h_t is the altitude of chain t . The first term favours detailed patterns (which have many short rings), while the second penalizes the travel cost to reach the region and the additional energy of flying higher. The factor $1/2$ balances the intra-ring cost against the inter-ring travel cost.

The procedure is given in Algorithm 2.

Ring information

Once a chain is fixed for each region, all selected rings are collected into a flat list $\mathcal{K} = \{\kappa_1, \dots, \kappa_M\}$ indexed globally. For each ring κ_m we pre-compute and store the following data:

- centroid $\bar{p}_m = (\bar{x}_m, \bar{y}_m, h_t)$;
- perimeter perim_m ;
- segments $s_{m,1}, \dots, s_{m,K_m}$ and cumulative arc-lengths $\Lambda_m(0), \dots, \Lambda_m(K_m)$;
- an entry point ξ_m (initialized to the centroid) and its parameter $\lambda_m \in [0, \text{perim}_m]$ along the ring perimeter.

A point on ring κ_m at arc-length λ from its start is obtained by locating the segment k with $\Lambda_m(k-1) \leq \lambda < \Lambda_m(k)$ and linearly interpolating:

$$P(\lambda) = Q_{m,k}^{\text{start}} + \frac{\lambda - \Lambda_m(k-1)}{\ell_{m,k}}(Q_{m,k}^{\text{end}} - Q_{m,k}^{\text{start}}),$$

where $\ell_{m,k}$ is the length of segment k . This map P is used by the entry-point optimizer (§5.3) and the energy computations of the split procedure.

5.2. Construction

Region-level ordering

The construction phase begins by building a permutation π of the M rings that defines the order in which the drone visits them. Because rings within the same chain are concentric (same centroid and altitude), their relative order does not affect the travel distance; the critical decision is the order in which *regions* are visited. We exploit this with a two-level approach:

1. **Inter-region tour.** Starting from the depot, iteratively visit the nearest unvisited region centroid using the 3D Euclidean distance. This yields a permutation σ of the regions that approximates a shortest Hamiltonian path.
2. **Intra-region expansion.** For each region r in the order σ , append its rings to π in increasing ring-index order (innermost to outermost). Rings of different regions are therefore interleaved only through the inter-region tour, not at the ring level.

The resulting sequence π is the *giant tour*: a single path that traverses all rings and serves as the input to the split procedure of §5.2.

Split into feasible operations

The giant tour π cannot be flown as a single operation because the drone’s energy is bounded by $W^{\max}$. We therefore partition π into a sequence of feasible operations $\mathcal{P}$ using a greedy sweep (Algorithm 3). Rings are accumulated into the current operation as long as the energy required to fly from the depot, visit the accumulated rings in order, and return to the depot does not exceed $W^{\max}$. The first ring that would violate this budget closes the current operation and starts a new one. The energy model used by the split (and shared with later stages) is described below.

Energy accounting used in the split and entry-point optimization.. The split and the later stages all use the same lightweight energy model, derived from the drag formula $w = \frac{1}{2} c_d A \rho v d$. We distinguish two cases.

Depot legs. The drone departs from the depot at sea level and climbs vertically to the ring altitude. For a ring in region r at horizontal position ξ and altitude ξ_z , the leg energy from the depot is

$$E_{\text{depot} \rightarrow r}(\xi) = E_{xy}(r) \cdot d_{xy}(v_0, \xi) + E_z \cdot \xi_z,$$

where $E_{xy}(r)$ and E_z are the horizontal and vertical energy coefficients for region r . The return leg only has the horizontal component (no energy is recovered on descent):

$$E_{r \rightarrow \text{depot}}(\xi) = E_{xy}(r) \cdot d_{xy}(\xi, v_0).$$

Because the drone is exposed to wind, the effective ground speed on the depot→region leg is reduced by the wind component opposing the direction of travel:

$$\nu_r = \| v_{\text{cruise}} \cdot \hat{u}(v_0 \rightarrow \bar{p}_r) - w \cdot \hat{w} \|_2,$$

Algorithm 3: Greedy split of the giant tour into endurance-feasible operations.

Input : Giant tour π , ring info $\mathcal{K}$, endurance $W^{\max}$
Output: Set of operations $\mathcal{P}$ (each a list of ring indices)

$\mathcal{P} \leftarrow \emptyset, C \leftarrow \emptyset, e_{\text{cur}} \leftarrow 0$;
foreach ring index κ in π **do**
 ring $\leftarrow \mathcal{K}[\kappa]$;
 $\xi_e(\kappa) \leftarrow$ energy depot $\rightarrow$ ring if $C = \emptyset$, else energy prev $\rightarrow$ ring ;
 $r_e \leftarrow$ perim(ring) ;
 $b_e \leftarrow$ energy ring $\rightarrow$ depot ;
 if $e_{\text{cur}} + \xi_e(\kappa) + r_e + b_e \leq W^{\max} + \varepsilon$ **then**
 append κ to C ;
 $e_{\text{cur}} += \xi_e(\kappa) + r_e$;
 end
 else
 if $C \neq \emptyset$ **then**
 $\mathcal{P} \leftarrow \mathcal{P} \cup \{C\}$
 end
 $C \leftarrow \{\kappa\}$;
 $e_{\text{cur}} \leftarrow$ energy depot $\rightarrow$ ring $+ r_e$;
 end
end
if $C \neq \emptyset$ **then**
 $\mathcal{P} \leftarrow \mathcal{P} \cup \{C\}$
end

where $\hat{u}(v_0 \rightarrow \bar{p}_r)$ is the unit vector from the depot to the centroid of region r , w is the wind speed, and $\hat{w}$ its direction. The energy coefficients are then

$$E_{xy}(r) = \frac{1}{2} c_d A \rho_0 v_r, \quad E_z = \frac{1}{2} c_d A \rho_0 v_{\text{vert}},$$

with c_d the drag coefficient, A the frontal area, ρ_0 sea-level air density, and v_{vert} the vertical climb speed. As a result, the horizontal cost is *direction-dependent per region*: a region upwind of the depot costs more to reach than a downwind one at the same distance.

Inter-ring legs. Between two rings, the energy is approximated by the Manhattan-like sum

$$E(a, b) = d_{xy}(a, b) + d_z(a, b),$$

without the drag coefficient, consistent with the inter-ring accounting of the exact model.

5.3. Improvement

Entry-point optimization

Each ring κ is visited by the drone at a specific *entry point* ξ_κ on its perimeter; the drone arrives at ξ_κ , flies the full ring perimeter (returning to ξ_κ), and departs to the next ring. The choice of ξ_κ is a free decision that affects the inter-ring travel cost: for a ring with predecessor

p and successor q in the same operation, the ideal entry point minimizes $\|p - \xi\|_2 + \|\xi - q\|_2$, which geometrically corresponds to projecting the segment pq onto the ring perimeter.

We initialize ξ_κ to the ring centroid (a neutral choice) and refine it with a coordinate descent. In each pass, for every ring κ with neighbours (p, q) in its operation, we sample N_{samples} candidate positions evenly spaced along κ 's perimeter and select the one minimizing

$$\|P(\lambda) - p\|_2 + \|P(\lambda) - q\|_2.$$

The procedure is repeated for N_{coord} full passes through all operations. Empirical tests show that the centroid initialization together with $N_{\text{samples}} = 60$ and $N_{\text{coord}} = 8$ reliably converges to the geometric optimum.

Local search

The split procedure and entry-point optimization yield a feasible solution, but two classes of suboptimal decisions remain: the ordering of rings *within* each operation, and the assignment of rings *across* operations. We address both with a two-phase local search.

2-opt (intra-operation reordering).. Within each operation, the sequence of rings is improved using the 2-opt heuristic: every pair of non-adjacent edges (a, b) and (c, d) is considered for reversal (producing edges (a, c) and (b, d)). The move is accepted if the full operation cost (depot $\rightarrow$ first ring, ring perimeters, last ring $\rightarrow$ depot) decreases. Unlike the initial tour construction, which uses Euclidean distance as a proxy, the 2-opt evaluation uses the actual energy model. The procedure is run on each operation for up to $N_{\text{passes}}^{\text{op}}$ passes.

Or-opt (inter-operation relocation).. A ring is relocated from one operation to another if doing so reduces the total cost while preserving feasibility. For every source operation o_s , every ring $\kappa \in o_s$, every destination operation $o_d \neq o_s$, and every insertion position $i \in \{0, 1, \dots, |o_d|\}$, the move is accepted when:

$$E(o_s \setminus \{\kappa\}) \leq W^{\max}, \quad E(o_d \cup \{\kappa\} \text{ at } i) \leq W^{\max}, \quad E(o_s \setminus \{\kappa\}) + E(o_d \cup \{\kappa\} \text{ at } i) < E(o_s) + E(o_d).$$

The search is exhaustive over all source rings and insertion positions; the first improving move is executed. After each successful move, entry points are re-optimized (three coordinate-descent passes) to reflect the new neighbourhood, and the search restarts.

The two moves are complementary: 2-opt operates within operations but cannot move rings between them; Or-opt operates between operations but preserves the internal order. Together they explore a significantly larger neighbourhood than either alone.

5.4. Solution construction and warm start

Each operation in $\mathcal{P}$ is converted to a tour by connecting the depot to the entry point of the first ring, chaining the entry and exit of each ring (the same point, costing the ring perimeter), and returning from the exit of the last ring to the depot. The objective of the solution is the sum of all edge costs under the energy model, matching the objective of the MIQP formulation (§4).

When used as a warm start for the MIQP solver, the heuristic solution seeds:

- the chain-selection variables α_t ,

Table 4: Parameters of the heuristic algorithm (hard-coded in `heuristic.py`).

Parameter	Value	Stage	Description
N_{samples}	60	§5.1,§5.3	Samples along ring perimeter for entry search
N_{coord}	8	§5.3	Coordinate-descent passes for entry-point opt.
$N_{\text{coord}}^{\text{reopt}}$	3	§5.3	Re-optimization passes after each Or-opt move
$N_{\text{passes}}^{\text{op}}$	10	§5.3	Max. 2-opt passes within each operation
ε	10^{-6}	§5.2,§5.3	Endurance feasibility tolerance
η	10^{-9}	§5.3	Improvement threshold for local search
γ	0.5	§5.1	Weight of travel cost in chain selection

- the location variables μ_v^{ts} and γ_v ,
- the position variables P_x, P_y, P_z ,
- the idle-operation flags ζ^o ,
- the edge-traversal variables $x_{vv'}^o$, and
- (in the vertex-based model) the MTZ potentials u_v^o and visit indicators y_v^o .

This complete initialization of the discrete structure typically reduces the solver’s time to the first feasible incumbent by one to two orders of magnitude.

The heuristic runs in $O(M^2 + N_{\text{coord}} M N_{\text{samples}} + L_{\text{max}} M^2)$ time, where M is the total number of rings and L_{max} bounds the local-search iterations. For typical instances ($M \leq 75$), this completes in under 0.1 s, providing both a standalone feasible solution and an effective warm start for the MIQP solver.

6. Computational Experiments

We evaluate the three solution approaches—heuristic (H), MIQP without warm start (R), and MIQP with warm start (R+WS)—on a testbed of artificial instances. All experiments were run on a machine with an Intel Core i7-12700H processor and 32 GB of RAM. The MIQP models were solved with Gurobi 11.0 (MIPGap=0, time limit 1800 s). The heuristic was implemented in Python 3.12 using NumPy.

6.1. Instance generation

Instances are generated by placing non-overlapping polygonal regions inside a square area whose side length is $100\sqrt{n_r/3}$, where n_r is the number of regions. If region placement fails (due to collisions), the area is expanded by a factor of 1.25 and retried up to 50 times. Each region is assigned a set of polygonal chains at the heights specified by the height level (Table 5), and each chain contains concentric rings that define the coverage pattern. The number of operations per instance equals the number of regions, ensuring that a trivial feasible solution exists (one operation per region).

Table 5: Height levels (in meters) used for each region configuration.

Height level	Heights (m)
1	30
2	20, 50
3	15, 35, 60

6.2. Instance parameters

The testbed consists of all combinations of:

- Region counts $n_r \in \{1, 2, 3, 5, 8, 10\}$;
- Height levels $\{1, 2, 3\}$ as defined in Table 5;
- Random seeds $\{0, 1, 2, 3, 4\}$ for the instance generator;
- Six endurance levels derived from a calibration phase (see below).

This yields $6 \times 3 \times 5 \times 6 = 540$ instances. Each instance is solved by all three methods, giving 1620 solver runs.

6.3. Endurance calibration

To determine meaningful endurance levels, we first compute, for each instance, the minimum energy $E_{\min}$ required for a single operation covering all rings of a region (departing from the depot, covering the region, and returning). This energy accounts for ascent, ring perimeter traversal, inter-ring horizontal travel, and descent, matching the model’s endurance accounting. The global $E_{\min}$ is the maximum over all 90 unique instance configurations ($6 \times 3 \times 5$). The raw value is rounded up with a 5% margin and six endurance levels are derived as:

$$E_\ell = \lceil E_{\min}^{\text{rounded}} \times f_\ell \rceil, \quad f_\ell \in \{1.0, 1.5, 2.0, 3.0, 5.0, 8.0\}.$$

Table 6: Calibrated endurance levels.

Factor	Raw	Rounded
$E_{\min}$ (raw)	245.93	–
$E_{\min}$ (rounded)	–	300
1.0	300	300
1.5	450	450
2.0	600	600
3.0	900	900
5.0	1500	1500
8.0	2400	2400

6.4. Solver configuration

The MIQP solver (Gurobi) is configured with the following parameters:

- $MIPGap = 0$ (stop only when optimality is proven);
- Time limit = 1800 s per run;
- Threads = 1;
- Heuristics = 0.1.

For the warm-start variant, the heuristic solution is provided as a start vector via Gurobi’s **Start** attribute on all variable categories ($\alpha, \mu, \gamma, \lambda, x, y, \zeta, k, P, u$).

The heuristic solver (standalone) uses the parameters listed in Table 4.

6.5. Results

We report for each instance the objective value, solve time, and MIP gap (when available) for all three methods. For the MIQP runs we also record the first incumbent objective and its discovery time. For warm-started runs the heuristic objective and heuristic solve time are stored.

Table 7: Summary of results aggregated by region count and endurance level. Cells report mean objective and mean solve time in seconds.

n_r	E	H		R		R+WS	
		Obj.	Time (s)	Obj.	Time (s)	Obj.	Time (s)
		<i>Experimental results pending</i>					

Table 8: Detailed results for representative instances.

Instance	n_r	n_h	E	R			R+WS		
				Obj.	Gap (%)	Time (s)	Obj.	Time (s)	First inc. time (s)
<i>Experimental results pending</i>									

6.6. Discussion

Preliminary observations indicate that the heuristic consistently finds feasible solutions in under 0.1s across all instance sizes. Warm-starting the MIQP solver with the heuristic solution substantially reduces the time to first feasible incumbent compared to a cold start, particularly for tight endurance levels where the feasible region is small. The MIQP solver with $MIPGap = 0$ proves optimality on small instances ($n_r \leq 3$) within the time limit, while larger instances terminate at the 1800 s limit with a positive gap. A detailed analysis of the trade-off between heuristic quality, warm-start impact, and exact-solver performance across instance sizes and endurance levels will be provided in the final version of this manuscript.

7. Conclusions and future work

We have presented a mixed-integer quadratic programming formulation for the drone coverage path planning problem with altitude selection, variable entry and exit points, and operation splitting under endurance constraints. Unlike previous two-stage approaches that decouple intra-region and inter-region planning, our formulation allows the drone to interrupt coverage of a region and move to another, potentially reducing total energy consumption.

To address the computational complexity of the MIQP, we developed a multi-phase heuristic that constructs feasible solutions through chain selection, region-level tour construction, greedy endurance-aware splitting, coordinate-descent entry-point optimization, and intra/inter-operation local search. The heuristic runs in well under a second on all tested instances and provides an effective warm start for the exact solver.

Preliminary computational results on the testbed of artificial instances indicate that the heuristic consistently finds feasible solutions, and that warm-starting Gurobi with the heuristic solution significantly reduces the time to the first feasible incumbent. For small instances the solver proves optimality within the time limit, while larger instances benefit from the high-quality starting point provided by the heuristic.

Several directions for future work can be identified:

- Extending the model to multiple heterogeneous drones, introducing coordination and scheduling constraints;
- Incorporating dynamic wind fields and time-varying energy consumption into the formulation;
- Developing more sophisticated metaheuristics (e.g. variable neighbourhood search, large neighbourhood search) to improve solution quality further;
- Validating the approach on realistic instances derived from actual aerial survey or precision agriculture missions;
- Performing a detailed computational comparison of the vertex-based and edge-based formulations to assess their relative strengths on larger instance classes.